

```

6      for(i=1; i<=n; i++)
7      {
8          nut=nut+i;
9          if(nut==t)
10         {
11             nut=nut-1;
12         }
13     }
14     printf("%lld",nut%1000000007);
15     return 0;
16 }
17

```

	Input	Expected	Got	
✓	2 2	3	3	✓
✓	2 1	2	2	✓
✓	3 3	5	5	✓

Passed all tests! ✓

5

Explanation 2

$2 + 3 = 5$, is the best case for maximum nutrients.

Answer: (penalty regime: 0 %)

```
1  #include<stdio.h>
2  int main()
3  {
4      long long int n,t,i,nut=0;
5      scanf("%lld %lld",&n,&t);
6      for(i=1;i<=n;i++)
7      {
8          nut=nut+i;
9          if(nut==t)
10         {
11             nut=nut-1;
12         }
13     }
14     printf("%lld",nut%1000000007);
15     return 0;
16 }
17 }
```

2

1

Sample Output 1

2

Explanation 1

1. Cannot use item 1 because $k = 1$ and $sum \equiv k$ has to be avoided at any time.
2. Hence, max total is achieved by $sum = 0 + 2 = 2$.

Sample Case 2

Sample Input For Custom Testing

Sample Input 2

3

3

Sample Output 2

Input Format For Custom Testing

The first line contains an integer, n , that denotes the number of food items.

The second line contains an integer, k , that denotes the unhealthy number.

Sample Input 0

2

2

Sample Output 0

3

Explanation 0

The following sequence of $n = 2$ food items:

1. Item 1 has 1 macronutrients.
2. $1 + 2 = 3$; observe that this is the max total, and having avoided having exactly $k = 2$ macronutrients.

Sample Input 1

the increasing order of their value. Compute the highest total of macronutrients that can be prescribed to a patient, without the sum matching the given 'unhealthy' number.

Here's an illustration:

Given 4 food items (hence value: 1,2,3 and 4), and the unhealthy sum being 6 macronutrients, on choosing items 1, 2, 3 -> the sum is 6, which matches the 'unhealthy' sum. Hence, one of the three needs to be skipped. Thus, the best combination is from among:

- $2 + 3 + 4 = 9$
- $1 + 3 + 4 = 8$
- $1 + 2 + 4 = 7$

Since $2 + 3 + 4 = 9$, allows for maximum number of macronutrients, 9 is the right answer.

Complete the code in the editor below. It must return an integer that represents the maximum total of macronutrients, modulo 1000000007 ($10^9 + 7$).

It has the following:

n : an integer that denotes the number of food items

k : an integer that denotes the unhealthy number

Constraints

- $1 \leq n \leq 2 \times 10^9$
- $1 \leq k \leq 4 \times 10^{15}$

```

13     }
14 }
15 if(y==1)
16 {
17     printf("true");
18 }
19 else
20 {
21     printf("false");
22 }
23 return 0;
24 }

```

	Input	Expected	Got	
✓	6	true	true	✓
✓	89	true	true	✓
✓	25	false	false	✓

Passed all tests! ✓

A nutritionist is labeling all the best power foods in the market. Every food item arranged in a single line, will have a value beginning from 1 and increasing by 1 for each, until all items have a value associated with them. An item's value is the same as the number of macronutrients it has. For example, food item with value 1 has 1 macronutrient, food item with value 2 has 2 macronutrients, and incrementing in this fashion.

The nutritionist has to recommend the best combination to patients, i.e. maximum total of macronutrients. However, the nutritionist must avoid prescribing a particular sum of macronutrients (an 'unhealthy' number), and this sum is known. The nutritionist chooses food items in

Explanation:

We get 68 after rotating 89, 86 is a valid number and $86! = 89$.

Example 3:

11 -> 11

Input: 11

Output: false

Explanation:

We get 11 after rotating 11, 11 is a valid number but the value remains the same, thus 11 is not a confusing number.

Note:

1. $0 \leq N \leq 10^9$
2. After the rotation we can ignore leading zeros, for example if after rotation we have 0008 then this number is considered as just 8.

Answer: (penalty regime: 0 %)

```
1 #include<stdio.h>
2 int main()
3 {
4     int n,x,y=1;
5     scanf("%d",&n);
6     while(n!=0 && y==1)
7     {
8         x=n%10;
9         n=n/10;
10        if(x==2 || x==3 || x==4 || x==7)
11        {
12            y++;
```

	Input	Expected	Got	
✓	5 10 15 20 25 30 35 40 45 50	5	5	✓

Passed all tests! ✓

Given a number N, return true if and only if it is a *confusing number*, which satisfies the following condition:

We can rotate digits by 180 degrees to form new digits. When 0, 1, 6, 8, 9 are rotated 180 degrees, they become 0, 1, 9, 8, 6 respectively. When 2, 3, 4, 5 and 7 are rotated 180 degrees, they become invalid. A *confusing number* is a number that when rotated 180 degrees becomes a **different** number with each digit valid.

Example 1:

6 -> 9

Input: 6

Output: true

Explanation:

We get 9 after rotating 6, 9 is a valid number and 9!=6.

Example 2:

89 -> 68

Input: 89

Output: true

Output:

5

Explanation:

The numbers meeting the criteria are 5, 15, 25, 35, 45.

Answer: (penalty regime: 0 %)

```
1 #include<stdio.h>
2 int main()
3 {
4     int n,x=0;
5     while(scanf("%d",&n)==1)
6     {
7         if(n%2!=0)
8         {
9             x++;
10        }
11    }
12    printf("%d",x);
13    return 0;
14
15 }
```

A set of N numbers (separated by one space) is passed as input to the program. The program must identify the count of numbers where the number is odd number.

Input Format:

The first line will contain the N numbers separated by one space.

Boundary Conditions:

$3 \leq N \leq 50$

The value of the numbers can be from -99999999 to 99999999

Output Format:

The count of numbers where the numbers are odd numbers.

Example Input / Output 1:

Input:

5 10 15 20 25 30 35 40 45 50

```
9         r = r + 1,  
10     }  
11     printf("%d",r);  
12 }
```

	Input	Expected	Got	
✓	10	4	4	✓
✓	5	3	3	✓
✓	20	5	5	✓
✓	500	9	9	✓
✓	1000	10	10	✓

Passed all tests! ✓

But as per Manisha only { \$1, \$2, \$3 } coins are enough to purchase any item ranging from \$1 to \$5. Hence minimum is 3. Likewise, denominations could also be { \$1, \$2, \$4 }. Hence answer is still 3.

Answer: (penalty regime: 0 %)

```
1 #include<stdio.h>
2 int main()
3 {
4     int N,r=0;
5     scanf("%d",&N);
6     while(N!=0)
7     {
8         N=N/2;
9         r=r+1;
10    }
11    printf("%d",r);
12 }
```

	Input	Expected	Got	
✓	10	4	4	✓
✓	5	3	3	✓

4

Sample Input 2:

5

Sample Output 2:

3

Explanation:

For test case 1, N=10.

According to Manish {\$1, \$2, \$3,... \$10} must be distributed.

But as per Manisha only {\$1, \$2, \$3, \$4} coins are enough to purchase any item ranging from \$1 to \$10. Hence minimum is 4. Likewise denominations could also be {\$1, \$2, \$3, \$5}. Hence answer is still 4.

For test case 2, N=5.

According to Manish {\$1, \$2, \$3, \$4, \$5} must be distributed.

task is to help Manisha come up with a minimum number of denominations for any arbitrary max price in Philaland.

Input Format

Contains an integer N denoting the maximum price of the item present on Philaland.

Output Format

Print a single line denoting the minimum number of denominations of coins required.

Constraints

$$1 \leq T \leq 100$$

$$1 \leq N \leq 5000$$

Refer the sample output for formatting

Sample Input 1:

10

Sample Output 1:

```

14 {
15     sum=sum+2;
16 }
17 n=n/10;
18 }
19 printf("%d",sum);
20 return 0;
21 }

```

	Input	Expected	Got	
✓	630	2	2	✓
✓	1288	4	4	✓

Passed all tests! ✓

The problem solvers have found a new Island for [coding](#) and named it as Philaland. These smart people were given a task to make a purchase of items at the Island easier by distributing various coins with different values. Manish has come up with a solution that if we make coins category starting from \$1 till the maximum price of the item present on Island, then we can purchase any item easily. He added the following example to prove his point.

Let's suppose the maximum price of an item is 5\$ then we can make coins of {\$1, \$2, \$3, \$4, \$5} to purchase any item ranging from \$1 till \$5.

Now Manisha, being a keen observer suggested that we could actually minimize the number of coins required and gave following distribution {\$1, \$2, \$3}. According to him any item can be purchased one time ranging from \$1 to \$5. Everyone was impressed with both of them. Your

Sample Case 1

Sample Input

1288

Sample Output

4

Explanation

Add the holes count for each digit, 1, 2, 8, 8. Return $0 + 0 + 2 + 2 = 4$.

Answer: (penalty regime: 0 %)

```
1 #include<stdio.h>
2 int main()
3 {
4     int rem,n,sum=0;
5     scanf("%d",&n);
6     while (n>0)
7     {
8         rem=n%10;
9         if(rem == 0 || rem == 4 || rem == 6 || rem == 9)
10        {
11            sum=sum+1;
12        }
13        else if(rem==8)
```

Given a number, you must determine the sum of the number of holes for all of its digits. For example, the number 819 has 3 holes.

Complete the program, it must return an integer denoting the total number of holes in num.

Constraints

$1 \leq \text{num} \leq 10^9$

Input Format For Custom Testing

There is one line of text containing a single integer num, the value to process.

Sample Input

630

Sample Output

2

Explanation

Add the holes count for each digit, 6, 3 and 0. Return $1 + 0 + 1 = 2$.

```

20         printf("Yes\n");
21     }
22     i++;
23 }
24 return 0;
25 }

```

	Input	Expected	Got	
✓	3	Yes	Yes	✓
	1	Yes	Yes	
	6	No	No	
	7			

Passed all tests! ✓

You are designing a poster which prints out numbers with a unique style applied to each of them. The styling is based on the number of closed paths or holes present in a given number.

The number of holes that each of the digits from 0 to 9 have are equal to the number of closed paths in the digit. Their values are:

1, 2, 3, 5, and 7 = 0 holes.

0, 4, 6, and 9 = 1 hole.

8 = 2 holes.

6

7

Output

Yes

Yes

No

Answer: (penalty regime: 0 %)

```
1 #include<stdio.h>
2 int main()
3 {
4     int T,i=0,n,t;
5     scanf("%d",&T);
6     while(i<T)
7     {
8         scanf("%d",&n);
9         t=n/4;
10        if(t%2==0 && n%2==0)
11        {
12            printf("No\n");
13        }
14        else if(t%2==1 && n%2==1)
15        {
16            printf("No\n");
17        }
18        else
19        {
```

Alice and Bob are playing a game called "Stone Game". Stone game is a two-player game. Let N be the total number of stones. In each turn, a player can remove either one stone or four stones. The player who picks the last stone, wins. They follow the "Ladies First" norm. Hence Alice is always the one to make the first move. Your task is to find out whether Alice can win, if both play the game optimally.

Input Format

First line starts with T , which is the number of test cases. Each test case will contain N number of stones.

Output Format

Print "Yes" in the case Alice wins, else print "No".

Constraints

$$1 \leq T \leq 1000$$

$$1 \leq N \leq 10000$$

Sample Input and Output

Input

3

1

```
29     break;
30     case 6:
31         printf("Friday");
32         break;
33     case 7:
34         printf("Saturday");
35         break;
36     case 8:
37         printf("Kryptonday");
38         break;
39     case 9:
40         printf("Coluday");
41         break;
42     case 10:
43         printf("Dexamday");
44         break;
45 }
46 }
```

	Input	Expected	Got	
✓	7	Kryptonday	Kryptonday	✓
✓	1	Monday	Monday	✓

Passed all tests! ✓

7

Example Output

Kryptonday

Example Input

1

Example Output Monday

Answer: (penalty regime: 0 %)

```
1 #include<stdio.h>
2 int main()
3 {
4     int n,d;
5     scanf("%d",&n);
6     if(n<296)
7         d=n;
8     else
9         d=n-296;
10    d%=10;
11    d=d+1;
12    d%=10;
13    switch(d)
14    {
15        case 1:
16            printf("Sunday");
17            break;
18        case 2:
19            printf("Monday");
20            break;
21        case 3:
22            printf("Tuesday");
23            break;
24        case 4:
25            printf("Wednesday");
26            break;
27        case 5:
```

✓	T 10 20	200	200	✓
✓	S 30 40	600	600	✓
✓	B 2 11	0	0	✓
✓	R 10 30	300	300	✓
✓	S 40 50	1000	1000	✓

Passed all tests! ✓

Superman is planning a journey to his home planet. It is very important for him to know which day he arrives there. They don't follow the 7-day week like us. Instead, they follow a 10-day week with the following days: Day Number Name of Day 1 Sunday 2 Monday 3 Tuesday 4 Wednesday 5 Thursday 6 Friday 7 Saturday 8 Kryptonday 9 Coluday 10 Daxamday Here are the rules of the calendar: • The calendar starts with Sunday always. • It has only 296 days. After the 296th day, it goes back to Sunday. You begin your journey on a Sunday and will reach after n. You have to tell on which day you will arrive when you reach there.

Input format: •

Contain a number n ($0 < n$)

Output format: Print the name of the day you are arriving on

Example Input

- First is output of area of rectangle
- Then, output of area of triangle
- Then output of area square
- Finally, something random, so we print 0

Answer: (penalty regime: 0 %)

```

1  #include<stdio.h>
2  int main()
3  {
4      int a,b;
5      char c;
6      scanf("%c%d%d",&c,&a,&b);
7      switch(c)
8      {
9          case 'R':
10             printf("%d",a*b);
11             break;
12          case 'S':
13             printf("%.0f", (0.5)*a*b);
14             break;
15          case 'T':
16             printf("%d",a*b);
17             break;
18          default:
19             printf("0");
20     }
21 }
```

Input	Expected	Got
-------	----------	-----

100

Sample Input 4

G

8

8

Sample Output 4

0

Sample Input

C

9

10

Sample Output 4

0

Explanation:

Sample Output 1

200

Sample Input 2

S

30

40

Sample Output 2

600

Sample Input 3

R

10

10

Sample Output 3

- When he says square, he is actually referring to a triangle.
- When he says triangle he is referring to a rectangle
- And when he is confused, he just says something random. At this point, all you can do is say 0.

Help Suppandi by printing the correct answer in an integer.

Input Format

- Name of shape (always in upper case R à Rectangle, S à Square, T à Triangle)
- Length of 1 side
- Length of other side

Note: In case of triangle, you can consider the sides as height and length of base

Output Format

- Print the area of the shape.

Sample Input 1

T
10
20

```

39         m=31+28+31+30+31+30+31+31+30+d;
40         break;
41     case 12:
42         m=30+28+31+30+31+30+31+31+30+31+d;
43         break;
44     }
45     if(m>58&& y%4==0)
46     {
47         printf("%d",m+1);
48     }
49     else
50     {
51         printf("%d",m);
52     }
53     return 0;
54 }

```

	Input	Expected	Got	
✓	18 6 2020	170	170	✓

Passed all tests! ✓

Suppandi is trying to take part in the local village math quiz. In the first round, he is asked about shapes and areas. Suppandi, is confused, he was never any good at math. And also, he is bad at remembering the names of shapes. Instead, you will be helping him [calculate the area](#) of shapes.

When he says rectangle he is actually referring to a square.

```

1 #include<stdio.h>
2 int main()
3 {
4     int d,h,y,m;
5     scanf("%d\n%d\n%d",&d,&h,&y);
6     switch(h)
7     {
8         case 1:
9             m=d;
10            break;
11        case 2:
12            m=31+d;
13            break;
14        case 3:
15            m=31+28+d;
16            break;
17        case 4:
18            m=31+28+31+d;
19            break;
20        case 5:
21            m=31+28+31+30+d;
22            break;
23        case 6:
24            m=31+28+31+30+31+d;
25            break;
26        case 7:
27            m=31+28+31+30+31+30+d;
28            break;
29        case 8:
30            m=31+28+31+20+31+30+31+d;
31            break;
32        case 9:
33            m=31+28+31+30+31+30+31+31+d;
34            break;
35        case 10:
36            m=31+28+31+30+31+30+31+31+31+d;
37            break;
38        case 11:
39            m=31+28+31+30+31+30+31+31+31+30+d;

```

Some data sets specify dates using the year and day of year rather than the year, month, and day of month. The day of year (DOY) is the sequential day number starting with day 1 on January 1st.

There are two calendars - one for normal years with 365 days, and one for leap years with 366 days. Leap years are divisible by 4. Centuries, like 1900, are not leap years unless they are divisible by 400. So, 2000 was a leap year.

To find the day of year number for a standard date, scan down the Jan column to find the day of month, then scan across to the appropriate month column and read the day of year number. Reverse the process to find the standard date for a given day of year.

Write a program to print the Day of Year of a given date, month and year.

Sample Input 1

18
6
2020

Sample Output 1

170

Answer: (penalty regime: 0 %)

```

3  {
4      int n;
5      char ch;
6      scanf("%c %d",&ch,&n);
7      if((n%2!=0 && ch%2!=0) || (n%2==0 && ch%2==0))
8          printf("The square is black.");
9      else if((n%2!=0 && ch%2==0) || (n%2==0 && ch%2!=0))
10         printf("The square is white.");
11     return 0;
12
13
14 }

```

	Input	Expected	Got	
✓	a 1	The square is black.	The square is black.	✓
✓	d 5	The square is white.	The square is white.	✓

Passed all tests! ✓

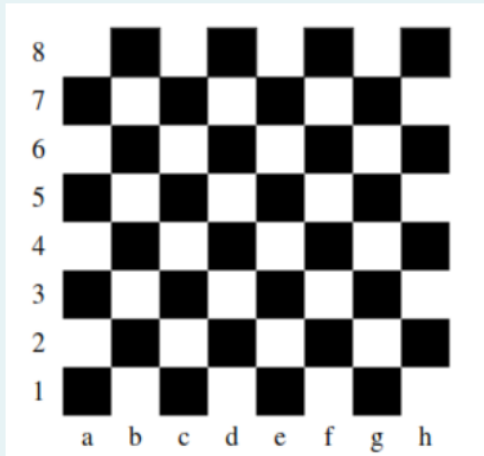
d 5

Sample Output 2

The square is white.

Answer: (penalty regime: 0 %)

```
1 #include<stdio.h>
2 int main()
3 {
4     int n;
5     char ch;
6     scanf("%c %d",&ch,&n);
7     if((n%2!=0 && ch%2!=0) || (n%2==0 && ch%2==0))
8     printf("The square is black.");
9     else if((n%2!=0 && ch%2==0) || (n%2==0 && ch%2!=0))
10    printf("The square is white.");
11    return 0;
12
13
14 }
```



Write a program that reads a position from the user. Use an if statement to determine if the column begins with a black square or a white square. Then use modular arithmetic to report the color of the square in that row. For example, if the user enters a1 then your program should report that the square is black. If the user enters d5 then your program should report that the square is white. Your program may assume that a valid position will always be entered. It does not need to perform any error checking.

Sample Input 1

a 1

Sample Output 1

The square is black.

Sample Input 2

```

30     case 3:
31         printf("Pig");
32         break;
33     case 4:
34         printf("Rat");
35         break;
36     case 5:
37         printf("Ox");
38         break;
39     case 6:
40         printf("Tiger");
41         break;
42     case 7:
43         printf("Hare");
44         break;
45     }
46     return 0;
47 }

```

	Input	Expected	Got	
✓	2004	Monkey	Monkey	✓
✓	2010	Tiger	Tiger	✓

Passed all tests! ✓

Positions on a chess board are identified by a letter and a number. The letter identifies the column, while the number identifies the row, as shown below:

Sample Output 2

Tiger

Answer: (penalty regime: 0 %)

```
1 #include<stdio.h>
2 int main()
3 {
4     int n,a;
5     scanf("%d",&n);
6     a=n%12;
7     switch(a)
8     {
9         case 8:
10            printf("Dragon");
11            break;
12         case 9:
13            printf("Snake");
14            break;
15         case 10:
16            printf("Horse");
17            break;
18         case 11:
19            printf("Sheep");
20            break;
21         case 0:
22            printf("Monkey");
23            break;
24         case 1:
25            printf("Rooster");
26            break;
27         case 2:
28            printf("Dog");
29            break;
```

2003	Sheep
2004	Monkey
2005	Rooster
2006	Dog
2007	Pig
2008	Rat
2009	Ox
2010	Tiger
2011	Hare

Write a program that reads a year from the user and displays the animal associated with that year. Your program should work correctly for any year greater than or equal to zero, not just the ones listed in the table.

Sample Input 1

2004

Sample Output 1

Monkey

Sample Input 2

2010

```
31     break;
32     default:
33         printf("The number of sides is not supported.");
34         break;
35     return 0;
36 }
37 }
```

	Input	Expected	Got	
✓	3	Triangle	Triangle	✓
✓	7	Heptagon	Heptagon	✓
✓	11	The number of sides is not supported.	The number of sides is not supported.	✓

Passed all tests! ✓

The Chinese zodiac assigns animals to years in a 12-year cycle. One 12-year cycle is shown in the table below. The pattern repeats from there, with 2012 being another year of the Dragon, and 1999 being another year of the Hare.

Year	Animal
2000	Dragon
2001	Snake
2002	Horse

Sample Output 3

The number of sides is not supported.

Answer: (penalty regime: 0 %)

```
1 #include<stdio.h>
2 int main()
3 {
4     int n;
5     scanf("%d",&n);
6     switch(n)
7     {
8         case 3:
9             printf("Triangle");
10            break;
11            case 4:
12                printf("Quadrilateral");
13                break;
14            case 5:
15                printf("Pentagon");
16                break;
17            case 6:
18                printf("Hexagon");
19                break;
20            case 7:
21                printf("Heptagon");
22                break;
23            case 8:
24                printf("Octagon");
25                break;
26            case 9:
27                printf("Nonagon");
28                break;
29            case 10:
30                printf("Decagon");
```

Write a program that determines the name of a shape from its number of sides. Read the number of sides from the user and then report the appropriate name as part of a meaningful message. Your program should support shapes with anywhere from 3 up to (and including) 10 sides. If a number of sides outside of this range is entered then your program should display an appropriate error message.

Sample Input 1

3

Sample Output 1

Triangle

Sample Input 2

7

Sample Output 2

Heptagon

Sample Input 3

11

```

3  {
4      int n;
5      float rp,dis,tot;
6      scanf("%d",&n);
7      rp=n*3.49;
8      dis=rp*0.6;
9      tot= rp-dis;
10     printf("Regular price: %.2f\nDiscount: %.2f\nTotal: %.2f\n",rp,dis,tot);
11     return 0;
12 }

```

	Input	Expected	Got	
✓	10	Regular price: 34.90 Discount: 20.94 Total: 13.96	Regular price: 34.90 Discount: 20.94 Total: 13.96	✓

Passed all tests! ✓

A bakery sells loaves of bread for \$3.49 each. Day old bread is discounted by 60 percent. Write a program that begins by reading the number of loaves of day old bread being purchased from the user. Then your program should display the regular price for the bread, the discount because it is a day old, and the total price. Each of these amounts should be displayed on its own line with an appropriate label. All of the values should be displayed using two decimal places.

Input Format

Read the number of day old loaves.

Output Format

First line, print Regular price: price

Second line, print Discount: discount

Third line, print Total: total

Note: All of the values should be displayed using two decimal places.

Sample Input 1

10

Sample Output 1

Regular price: 34.90

Discount: 20.94

Total: 13.96

Answer: (penalty regime: 0 %)

1	#include<stdio.h>
2	int main()
3	{
4	int n;
5	float rp,dis,tot;
6	scanf("%d",&n);
7	rp=n*3.49;
8	dis=rp*0.6;
9	tot= rp-dis;

```

1  #include<stdio.h>
2  int main()
3  {
4      int a,b,sum,diff,pro,q,r;
5      scanf("%d%d",&a,&b);
6      sum=a+b;
7      diff=a-b;
8      pro=a*b;
9      q=a/b;
10     r=a%b;
11     printf("%d\n%d\n%d\n%d\n",sum,diff,pro,q,r);
12     return 0;
13 }

```

	Input	Expected	Got	
✓	100	106	106	✓
	6	94	94	
		600	600	
		16	16	
		4	4	

Passed all tests! ✓

	Input	Expected	Got	
✓	5 6	167.64	167.64	✓

Passed all tests! ✓

Create a program that reads two integers, a and b, from the user. Your program should compute and display:

- The sum of a and b
- The difference when b is subtracted from a
- The product of a and b
- The quotient when a is divided by b
- The remainder when a is divided by b

Input Format

First line, read the first number.

Second line, read the second number.

Output Format

First line, print the sum of a and b

Second line, print the difference when b is subtracted from a

Third line, print the product of a and b

Fourth line, print the quotient when a is divided by b

Fifth line, print the remainder when a is divided by b

Sample

Input 1 100 6

Sample Output

106 94 600 16 4

Answer: (penalty regime: 0 %)

Many people think about their height in feet and inches, even in some countries that primarily use the metric system. Write a program that reads a number of feet from the user, followed by a number of inches. Once these values are read, your program should compute and display the equivalent number of centimeters.

Hint:

One foot is 12 inches.

One inch is 2.54 centimeters.

Input Format

First line, read the number of feet.

Second line, read the number of inches.

Output Format

In one line print the height in centimeters.

Note: All of the values should be displayed using two decimal places.

Sample Input 1

5 6

Sample Output 1

167.64

Answer: (penalty regime: 0 %)

```
1 #include<stdio.h>
2 int main()
3 {
4     float f,i,cmf;
5     scanf("%f%f",&f,&i);
6     cmf=f*12*2.54+i*2.54;
7     printf("%.2f",cmf);
8     return 0;
9 }
```

```

8     double d;
9     scanf("%d %ld %c %f %lf",&a,&b,&ch,&f,&d);
10    printf("%d\n%ld\n%c\n%.3f\n%.9lf\n",a,b,ch,f,d);
11    return 0;
12 }

```

	Input	Expected	Got	
✓	3 12345678912345 a 334.23 14049.30493	3 12345678912345 a 334.230 14049.304930000	3 12345678912345 a 334.230 14049.304930000	✓

Passed all tests! ✓

Write a program to print the [ASCII value](#) and the two adjacent characters of the given character.

Input

Input consists of the following space-separated values: *int*, *long*, *char*, *float*, and *double*, respectively.

Output Format

Print each element on a new line in the same order it was received as input. Note that the floating point value should be correct up to 3 decimal places and the double to 9 decimal places.

Sample Input

3 12345678912345 a 334.23 14049.30493

Sample Output

3
12345678912345
a
334.230
14049.304930000

Explanation

Print *int* **3**,
followed by *long* **12345678912345**,
followed by *char* **a**,
followed by *float* **334.23**,
followed by *double* **14049.30493**.

Answer: (penalty regime: 0 %)

1	#include<stdio.h>	
2	int main()	
3	{	
4	int a;	
5	long b;	
6	char ch;	
7	float f;	

- *Int* ("%d"): 32 Bit integer
 - *Long* ("%ld"): 64 bit integer
 - *Char* ("%c"): Character type
 - *Float* ("%f"): 32 bit real value
 - *Double* ("%lf"): 64 bit real value
-

Reading

To read a data type, use the following syntax:

```
scanf("`format_specifier`, &val)
```

For example, to read a *character* followed by a *double*:

```
char ch;
```

```
double d;
```

```
scanf("%c %lf", &ch, &d);
```

For the moment, we can ignore the spacing between format specifiers.

Printing

To print a data type, use the following syntax:

```
printf("`format_specifier`, val)
```

For example, to print a *character* followed by a *double*:

```
char ch = 'd';
```

```
double d = 234.432;
```

```
printf("%c %lf", ch, d);
```

Note: You can also use *cin* and *cout* instead of *scanf* and *printf*; however, if you are taking a million numbers as input and printing a million lines, it is faster to use *scanf* and *printf*.

```

7   scanf("%d%d%d",&m1,&m2,&m3);
8   avg= (m1+m2+m3)/3;
9   printf("%c\n",A);
10  printf("%d",avg);
11  return 0;
12  }

```

	Input	Expected	Got	
✓	A 3 4 6	A 4	A 4	✓
✓	T 7 3 8	T 6	T 6	✓
✓	R 0 100 99	R 66	R 66	✓

Passed all tests! ✓

Some C data types, their format specifiers, and their most common bit widths are as follows:

A
3 4 6

Sample Output 1 :

A
4

Sample Input 2 :

T
7 3 8

Sample Output 2 :

T
6

Answer: (penalty regime: 0 %)

```
1 #include<stdio.h>
2 int main()
3 {
4     char A;
5     int m1,m2,m3,avg;
6     scanf("%c",&A);
```

Write a program to input a name (as a single character) and marks of three tests as m1, m2, and m3 of a student considering all the three marks have been given in integer format.

Now, you need to calculate the average of the given marks and print it along with the name as mentioned in the output format section.

All the test marks are in integers and hence calculate the average in integer as well. That is, you need to print the integer part of the average only and neglect the decimal part.

Input format :

Line 1 : Name(Single character)

Line 2 : Marks scored in the 3 tests separated by single space.

Output format :

First line of output prints the name of the student.

Second line of the output prints the average mark.

Constraints

Marks for each student lie in the range 0 to 100 (both inclusive)

Sample Input 1 :

```

5 float c,d,addi,diffi;
6 scanf("%d%d",&a,&b);
7 scanf("%f%f",&c,&d);
8 addi= a+b;
9 diffi= a-b;
10 addf=c+d;
11 difff=c-d;
12 printf("%d %d\n",addi,diffi);
13 printf("%.1f %.1f\n",addf,difff);
14 return 0;
15 }

```

	Input	Expected	Got	
✓	10 4 4.0 2.0	14 6 6.0 2.0	14 6 6.0 2.0	✓
✓	20 8 8.0 4.0	28 12 12.0 4.0	28 12 12.0 4.0	✓

Passed all tests! ✓

10 4
4.0 2.0

Sample Output

14 6
6.0 2.0

Explanation

When we sum the integers **10** and **4**, we get the integer **14**. When we subtract the second number **4** from the first number **10**, we get **6** as their difference.

When we sum the floating-point numbers **4.0** and **2.0**, we get **6.0**. When we subtract the second number **2.0** from the first number **4.0**, we get **2.0** as their difference.

Answer: (penalty regime: 0 %)

1	#include<stdio.h>
2	int main()
3	{
4	int a,b,addint,diffint;
5	float c,d,addf,diff;
6	scanf("%d%d",&a,&b);
7	scanf("%f%f",&c,&d);
8	addint= a+b;
9	diffint= a-b;
10	addf=c+d;
11	diff=c-d;

1. Declare **4** variables: two of type int and two of type float.
2. Read **2** lines of input from stdin (according to the sequence given in the 'Input Format' section below) and initialize your **4** variables.
3. Use the **+** and **-** operator to perform the following operations:
 - o Print the sum and difference of two int variable on a new line.
 - o Print the sum and difference of two float variable rounded to one decimal place on a new line.

Input Format

The first line contains two integers.

The second line contains two floating point numbers.

Constraints

- $1 \leq \text{integer variables} \leq 10^4$
- $1 \leq \text{float variables} \leq 10^4$

Output Format

Print the sum and difference of both integers separated by a space on the first line, and the sum and difference of both float (scaled to **1** decimal place) separated by a space on the second line.

Sample Input

	Input	Expected	Got	
✓	C	C	C	✓

Passed all tests! ✓

Objective

The fundamental data types in c are int, float and char. Today, we're discussing int and float data types.

The printf() function prints the given statement to the console. The syntax is printf("format string",argument_list);. In the function, if we are using an integer, character, string or float as argument, then in the format string we have to write %d (integer), %c (character), %s (string), %f (float) respectively.

The scanf() function reads the input data from the console. The syntax is scanf("format string",argument_list);. For ex:
The scanf("%d",&number) statement reads integer number from the console and stores the given value in variable **number**.

To input two integers separated by a space on a single line, the command is scanf("%d %d", &n, &m), where **n** and **m** are the two integers.

Task

Your task is to take two numbers of [int data type](#), two numbers of float data type as input and output their sum:

Input Format

Take a character, **ch** as input.

Output Format

Print the character, **ch**.

Answer: (penalty regime: 0 %)

```
1 #include<stdio.h>
2 int main()
3 {
4     char ch;
5     scanf("%c",&ch);
6     printf("%c",ch);
7     return 0;
8 }
```

	Expected	Got	
✓	Hello, World!	Hello, World!	✓

Passed all tests! ✓

Objective

This challenge will help you to learn how to take a character, a string and a sentence as input in C.

To take a single character ***ch*** as input, you can use `scanf("%c", &ch);` and `printf("%c", ch)` writes a character specified by the argument `char` to stdout:

```
char ch;  
scanf("%c", &ch);  
printf("%c", ch);
```

This piece of code prints the character ***ch***.

Task

You have to print the character, ***ch***.

Objective

This is a simple challenge to help you practice printing to stdout.

We're starting out by printing the most famous computing phrase of all time! In the editor below, use either `printf` or `cout` to print the string ***Hello, World!*** to stdout.

Input Format

You do not need to read any input in this challenge.

Output Format

Print ***Hello, World!*** to stdout.

Sample Output

Hello, World!

Answer: (penalty regime: 0 %)

1	#include<stdio.h>
2	int main()
3	{
4	printf("Hello, World!");
5	return 0;